

AI Assisted Coding Lab ASS-13.4

Name: B. SATHWIK REDDY

Batch:14

2303A510B1

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Expected Output

```
[4, 8, 12, 16, 20]
```

PROMPT:

#Refactor this Python code to double numbers using a Pythonic method while allowing user input.

CODE:

OUTPUT:

```
Enter numbers separated by spaces: 2 4 6 8 10
Doubled numbers are: 4 8 12 16 20
PS C:\Users\sathw\.vscode>
```

Task 2: Improving Text Processing Code

Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyse the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",
"quality"]
```

```
message = ""
```

```
for w in words:
```

```
    message += w + " "
```

```
print (message. Strip ())
```

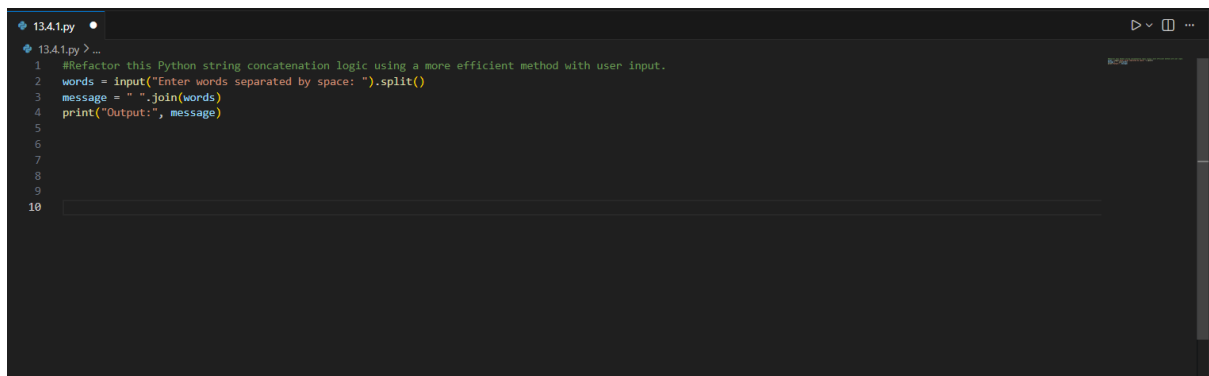
Expected Output

Refactoring with AI improves quality

PROMPT:

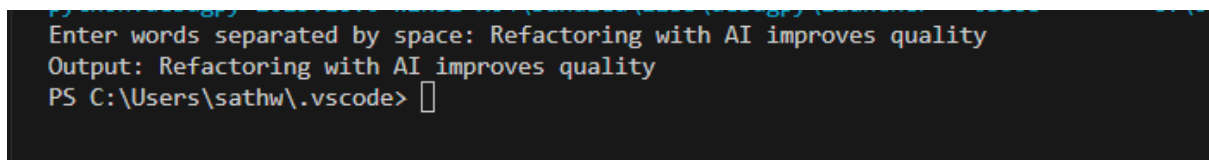
#Refactor this Python string concatenation logic using a more efficient method with user input.

CODE:

A screenshot of a code editor window titled '134.1.py'. The code is as follows:

```
1 #Refactor this Python string concatenation logic using a more efficient method with user input.
2 words = input("Enter words separated by space: ").split()
3 message = " ".join(words)
4 print("Output:", message)
5
6
7
8
9
10
```

OUTPUT:

A screenshot of a terminal window showing the execution of the program. The input and output are as follows:

```
Enter words separated by space: Refactoring with AI improves quality
Output: Refactoring with AI improves quality
PS C:\Users\sathw\.vscode>
```

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary

access methods.

- Ensure the behaviour remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
```

```
if "timeout" in config:
```

```
    print(config["timeout"])
```

```
else:
```

```
    print ("Default timeout used")
```

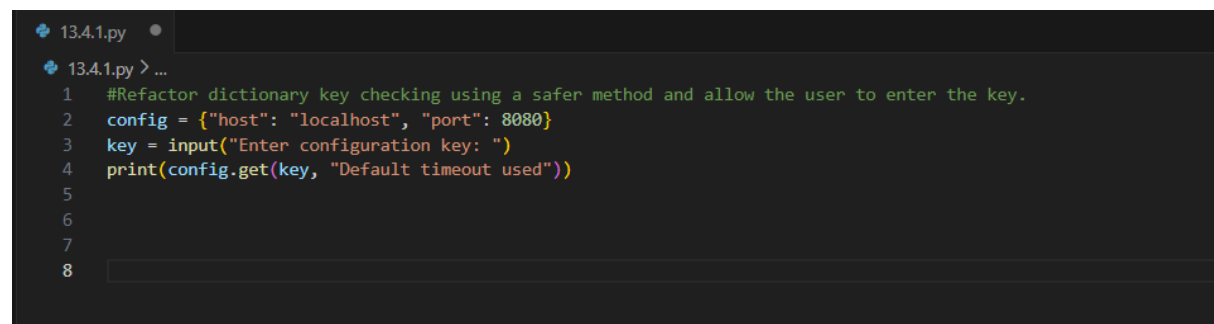
Expected Output

Default timeout used

PROMPT:

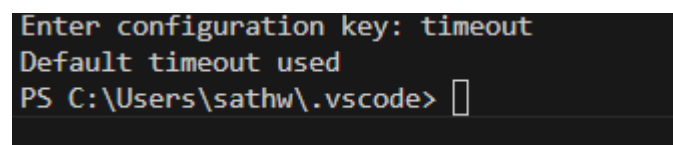
#Refactor dictionary key checking using a safer method and allow the user to enter the key.

CODE:

A screenshot of a code editor with a dark theme. The file name is '13.4.1.py'. The code is as follows:

```
1  #Refactor dictionary key checking using a safer method and allow the user to enter the key.
2  config = {"host": "localhost", "port": 8080}
3  key = input("Enter configuration key: ")
4  print(config.get(key, "Default timeout used"))
5
6
7
8
```

OUTPUT:

A screenshot of a terminal window with a dark background. The text shown is:

```
Enter configuration key: timeout
Default timeout used
PS C:\Users\sathw\.vscode>
```

Task 4: Refactoring Conditional Logic for

Scalability

Scenario

You are enhancing a utility module that performs different operations

based on user input.

Task Description

- Examine the multiple if–Elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
```

```
x, y = 10, 2
```

```
if action == "add":
```

```
    result = x + y
```

```
elif action == "subtract":
```

```
    result = x - y
```

```
elif action == "multiply":
```

```
    result = x * y
```

```
elif action == "divide":
```

```
    result = x / y
```

```
else:
```

```
    result = None
```

```
print(result)
```

Expected Output

5.0

PROMPT:

#Refactor this Python operation logic using dictionary mapping and allow the user to enter operation and numbers.

CODE:

```
13.4.1.py X
13.4.1.py > ...
1 #Refactor this Python operation logic using dictionary mapping and allow the user to enter operation and numbers.
2 def add(x, y):
3     return x + y
4
5 def subtract(x, y):
6     return x - y
7 def multiply(x, y):
8     return x * y
9 def divide(x, y):
10    if y == 0:
11        return "Error: Division by zero is not allowed."
12    return x / y
13 operations = {}
14 'add': add,
15 'subtract': subtract,
16 'multiply': multiply,
17 'divide': divide
18 }
19 def calculator():
20     operation = input("Enter the operation (add, subtract, multiply, divide): ")
21     if operation not in operations:
22         print("Invalid operation. Please try again.")
23         return
24     try:
25         num1 = float(input("Enter the first number: "))
26         num2 = float(input("Enter the second number: "))
27     except ValueError:
28         print("Invalid input. Please enter numeric values.")
29         return
30     result = operations[operation](num1, num2)
31     print(f"The result of {operation}ing {num1} and {num2} is: {result}")
32 calculator()
33
34
35
```

OUTPUT:

```
Enter the operation (add, subtract, multiply, divide): subtract
Enter the first number: 20
Enter the second number: 10
The result of subtracting 20.0 and 10.0 is: 10.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode> c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49393' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
The result of subtracting 20.0 and 10.0 is: 10.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode> c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49393' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode> c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49393' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49393' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
Enter the operation (add, subtract, multiply, divide): multiply
Enter the first number: 3
Enter the second number: 4
Enter the first number: 3
Enter the second number: 4
Enter the second number: 4
The result of multiplying 3.0 and 4.0 is: 12.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode> c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49393' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
PS C:\Users\sathw\.vscode> c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49675' '-.' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): divide
Enter the first number: 30
Enter the second number: 5
The result of dividing 30.0 and 5.0 is: 6.0
PS C:\Users\sathw\.vscode> ^
```

Task 5: Simplifying Search Logic in

Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops

to find elements.

Task Description

- Identify the explicit loop used for searching an item.

- Use AI assistance to refactor the logic into a more concise and readable form.

- Maintain the same output behaviour.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
```

```
found = False
```

```
for item in inventory:
```

```
    if item == "eraser":
```

```
        found = True
```

```
        break
```

```
print ("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

PROMPT:

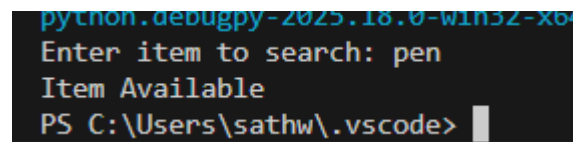
Refactor this Python list search logic using a more concise method and allow the user to enter the item.

CODE:

A screenshot of a VS Code editor window with a dark theme. The file name is '13.4.1.py'. The code is as follows:

```
1  #Refactor this Python list search logic using a more concise method and allow the user to enter the item.
2  inventory = ["pen", "notebook", "eraser", "marker"]
3
4  item = input("Enter item to search: ")
5
6  print("Item Available" if item in inventory else "Item Not Available")
7
8
9
```

OUTPUT:

A screenshot of a terminal window with a dark background. The text shown is:

```
python.debugpy-2025.18.0-win32-x64
Enter item to search: pen
Item Available
PS C:\Users\sathw\.vscode>
```